

LABORATORY RECORD

Course: Full Stack Web Development

Course Code: 23CM4121

Experiment No.: 3

Student Name: B.Charishma

Roll No.: a24126552067

Date: 30-08-2026

1. TITLE

Core JavaScript Logic, Array Manipulation, and Functional Programming

2. AIM

To implement functional programming patterns in JavaScript utilizing ES6 arrow functions, advanced array methods like filter and reduce, and dynamic DOM injection.

3. OBJECTIVE

The objectives of this experiment are:

- To declare constants and initialize array data structures.
- To compute aggregations using the `Array.prototype.reduce()` method.
- To implement conditional extractions using the `Array.prototype.filter()` method.
- To utilize ES6 destructuring and the `Math` utility object.
- To dynamically build and inject HTML strings into the Document Object Model.

4. THEORY

JavaScript treats functions as first-class citizens, meaning they can be passed as arguments, returned, and assigned to variables. ES6 arrow functions provide a concise syntax for writing function expressions.

Array methods like `map`, `filter`, and `reduce` are foundational for data transformation, avoiding the need for manual loops and mutable state, thus adhering to functional programming principles.

Application Flow:

Data Initialization (Array) -> Aggregation (Reduce) -> Extraction (Filter) -> Math Operations -> DOM InnerHTML Update

5. ALGORITHM / PROCEDURE

1. Define a constant array of integer marks representing student scores.
2. Create a function `calculateAverage` that leverages `reduce()` to sum the array.
3. Create an arrow function `filterHighScores` to return scores ≥ 80 .
4. Create a function `getMinMax` utilizing the spread operator `...arr` and `Math.max/min`.
5. Execute the functions and destructure the results into separate variables.
6. Log the intermediate calculations to the browser console.
7. Construct a template literal string embedding the calculated variables.
8. Inject the final string into a target DOM element using `document.getElementById().innerHTML`.

6. PROGRAM / SOURCE CODE

Source File: week3rd.html

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>JS Basics - Arrays & Functions</title>
</head>
<body>
  <h1>JavaScript Arrays & Functions Demonstration</h1>
  <div id="output"></div>

  <script>
    // Array Initialization
    const marks = [85, 92, 78, 64, 95, 88];

    // Function 1: Calculate Average
    const calculateAverage = function(arr) {
      const sum = arr.reduce((acc, curr) => acc + curr, 0);
      return (sum / arr.length).toFixed(2);
    }

    // Function 2: Filter High Scores (Marks >= 80)
    const filterHighScores = (arr) => arr.filter(score => score >= 80);

    // Function 3: Find Max and Min Marks
    function getMinMax(arr) {
      return {
        max: Math.max(...arr),
        min: Math.min(...arr)
      };
    }

    // Execution
    const avg = calculateAverage(marks);
    const topMarks = filterHighScores(marks);
    const { max, min } = getMinMax(marks);

    // Output to Browser Console
    console.log("Original Marks:", marks);
    console.log("Average Marks:", avg);
    console.log("Top Marks (>= 80):", topMarks);
    console.log(`Highest Mark: ${max}, Lowest Mark: ${min}`);

    // Output to Web Page DOM
    document.getElementById("output").innerHTML = `
      <p><strong>Original Marks:</strong> ${marks.join(", ")}</p>
      <p><strong>Average Marks:</strong> ${avg}</p>
      <p><strong>Top Scores (&ge; 80):</strong> ${topMarks.join(", ")}</p>
      <p><strong>Highest:</strong> ${max} | <strong>Lowest:</strong> ${min}</p>
    `;
  </script>

```

```
</body>
</html>
```

7. CODE EXPLANATION

Code / Syntax	Explanation
<code>arr.reduce((acc, curr) => acc + curr, 0)</code>	Executes a reducer function on each element, resulting in a single output value.
<code>arr.filter(score => score >= 80)</code>	Creates a new array with all elements that pass the test implemented by the provided function.
<code>Math.max(...arr)</code>	Uses the spread syntax to pass array elements as individual arguments to find the maximum value.
<code>const { max, min } = getMinMax(marks);</code>	Object destructuring assignment to unpack properties from an object into distinct variables.
<code>marks.join(', ')</code>	Creates and returns a new string by concatenating all of the elements in an array.

8. TEST CASES AND EXECUTION DATA

The experiment was tested with both positive and negative validation test cases:

Test Case	Input / Action	Expected Result	Actual Result	Status
TC-01	Average Calculation	Pass [85, 92, 78, 64, 95, 88] to calculateAverage	Returns exactly 83.67 as formatted average	Pass
TC-02	High Score Filtering	Pass same array to filterHighScores	Returns new array containing [85, 92, 95, 88]	Pass
TC-03	Spread Operator Max/Min	Execute getMinMax on array	Correctly identifies 95 as Max and 64 as Min	Pass
TC-04	Empty Array Execution	Provide [] to reduce and filter functions	Reduce returns 0, Max/Min return -Infinity/Infinity	Fail
TC-05	String Array Manipulation	Pass ['A', 'B', 'C'] to Math.max	Execution results in NaN due to type coercion failure	Fail

9. OUTPUT

Output 1: Dynamic Data Calculation and DOM Injection (Passed Output)

JavaScript Arrays & Functions Demonstration

Original Marks: 85, 92, 78, 64, 95, 88

Average Marks: 83.67

Top Scores (≥ 80): 85, 92, 95, 88

Highest: 95 | Lowest: 64

10. RESULT

Thus, the experiment was successfully implemented and verified across both positive functional and negative validation test cases.